

AG35-CET&AG35-EUT

QuecOpen(SDK) SMS 开发指导

LTE Standard 模块系列

版本：1.0

日期：2024-08-20

状态：受控文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2024，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2024.

文档历史

修订记录

版本	日期	作者	变更表述
-	2023-07-08	Chann YE	文档创建
1.0	2024-08-20	Lyndsay XIE	受控版本 1. 新增适用模块 AG35-EUT。 2. 基于 QuecOpen 方案统一命名，更新文档名称。

目录

文档历史	3
目录	4
表格索引	5
1 引言	6
2 SMS API	7
2.1. 头文件	7
2.2. 函数概览	7
2.3. 函数详解	8
2.3.1. ql_sms_init	8
2.3.2. ql_sms_deinit	8
2.3.3. ql_sms_set_service_center_addr	9
2.3.4. ql_sms_get_service_center_addr	9
2.3.5. ql_sms_send_msg	10
2.3.5.1. ql_sms_msg_t	10
2.3.5.2. QL_SMS_MSG_FORMAT_E	11
2.3.6. ql_sms_send_msg_async	11
2.3.6.1. ql_sms_msg_async_cb_f	12
2.3.7. ql_sms_set_msg_rcv_cb	13
2.3.7.1. ql_sms_msg_rcv_cb_f	13
2.3.7.2. ql_sms_timestamp_t	14
2.3.7.3. ql_sms_user_data_head_t	14
2.3.8. ql_sms_send_pdu	15
2.3.8.1. ql_sms_pdu_t	15
2.3.8.2. QL_SMS_PDU_FORMAT_E	16
2.3.9. ql_sms_send_pdu_async	16
2.3.9.1. ql_sms_pdu_async_cb_f	17
2.3.10. ql_sms_set_pdu_rcv_cb	17
2.3.10.1. ql_sms_pdu_rcv_cb_f	18
2.3.11. ql_sms_set_service_error_cb	18
2.3.11.1. ql_sms_service_error_cb_f	19
2.4. 错误码及恢复措施	19
3 示例	21
3.1. 获取短信中心号码	21
3.2. 发送文本短信	22
3.3. 接收文本短信	23
3.4. 发送 PDU 短信	26
3.5. 接收 PDU 短信	27
4 附录 参考文档及术语缩写	29

表格索引

表 1: 函数概览	7
表 2: 错误码及恢复措施	19
表 3: 参考文档	29
表 4: 术语缩写	29

Quectel
Confidential

1 引言

移远通信 AG35-CET 和 AG35-EUT 模块支持 QuecOpen[®]方案。QuecOpen[®]是基于 Linux 的嵌入式开发平台，可简化 IoV 应用的软件设计和开发过程。有关 QuecOpen[®]的详细信息，请参考文档 [1]。

本文档适用于 SDK 构建环境的 QuecOpen[®]方案，主要介绍 AG35-CET 和 AG35-EUT 模块的 SMS API 及相关示例。

Quectel
Confidential

2 SMS API

2.1. 头文件

SMS API 头文件为 `ql_sms.h`，位于 SDK 的 `ql-sysroots/usr/include/ql-sdk/` 目录下。若无特别说明，本文档所涉及头文件均在该目录下。

2.2. 函数概览

表 1：函数概览

函数	说明
<code>ql_sms_init()</code>	初始化 SMS 服务
<code>ql_sms_deinit()</code>	注销 SMS 服务
<code>ql_sms_set_service_center_addr()</code>	设置短信中心号码
<code>ql_sms_get_service_center_addr()</code>	获取短信中心号码
<code>ql_sms_send_msg()</code>	发送文本短信
<code>ql_sms_send_msg_async()</code>	异步发送文本短信
<code>ql_sms_set_msg_rcv_cb()</code>	设置文本短信接收回调函数
<code>ql_sms_send_pdu()</code>	发送 PDU 短信
<code>ql_sms_send_pdu_async()</code>	异步发送 PDU 短信
<code>ql_sms_set_pdu_rcv_cb()</code>	设置 PDU 短信接收回调函数
<code>ql_sms_set_service_error_cb()</code>	设置 SMS 服务异常回调函数

备注

若无特别说明，本文档所述函数均不支持并发调用，并且不能在相关回调函数中调用以上函数。

2.3. 函数详解

2.3.1. ql_sms_init

该函数用于初始化 SMS 服务。

- 函数原型

```
int ql_sms_init(void)
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败；错误码详见 *ql_type.h*。

备注

使用其他 SMS API 前，必须调用本函数初始化 SMS 服务。

2.3.2. ql_sms_deinit

该函数用于注销 SMS 服务。

- 函数原型

```
int ql_sms_deinit(void)
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败；错误码详见 *ql_type.h*。

备注

若不再使用 SMS，必须调用本函数注销 SMS 服务以释放资源。

2.3.3. ql_sms_set_service_center_addr

该函数用于设置短信中心号码。

- 函数原型

```
int ql_sms_set_service_center_addr(char *addr, int len)
```

- 参数

addr:

[In] 短信中心号码。

len:

[In] 短信中心号码长度。单位：字节。

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败；错误码详见 *ql_type.h*。

2.3.4. ql_sms_get_service_center_addr

该函数用于获取短信中心号码。

- 函数原型

```
int ql_sms_get_service_center_addr(char *addr, int len)
```

- 参数

addr:

[Out] 短信中心号码。

len:

[In] 短信中心号码长度。单位：字节。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败；错误码详见 *ql_type.h*。

2.3.5. ql_sms_send_msg

该函数用于发送文本短信。

- 函数原型

```
int ql_sms_send_msg(ql_sms_msg_t *p_msg)
```

- 参数

p_msg:

[In] 文本短信；详见第 2.3.5.1 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败；错误码详见 *ql_type.h*。

备注

为方便用户使用，当短信息格式为 GSM 7-bit (**QL_SMS_MSG_FORMAT_GSM_7BIT**) 时，用户输入 ISO8859-1 格式的短信息即可，无需转换成 GSM 7-bit 格式，该函数将自动进行格式转换。ISO8859-1 编码向下兼容 ASCII 编码。

2.3.5.1. ql_sms_msg_t

文本短信结构体定义如下：

```
typedef struct
{
    QL_SMS_MSG_FORMAT_E format;
    char addr[QL_SMS_MAX_ADDR_LENGTH + 1];
    int content_size;
    char content[QL_SMS_MAX_SEND_MSG_LENGTH];
} ql_sms_msg_t
```

● 参数

类型	参数	描述
QL_SMS_MSG_FORMAT_E	<i>format</i>	文本短信格式；详见第2.3.5.2章。
char	<i>addr</i>	发送短信时为接收方地址； 接收短信时为发送方地址。
int	<i>content_size</i>	文本短信长度。范围：0~1440；单位：字节。
char	<i>content</i>	文本短信内容

2.3.5.2. QL_SMS_MSG_FORMAT_E

文本短信格式枚举定义如下：

```
typedef enum
{
    QL_SMS_MSG_FORMAT_GSM_7BIT    = 0,
    QL_SMS_MSG_FORMAT_BINARY_DATA = 1,
    QL_SMS_MSG_FORMAT_UCS2        = 2,
} QL_SMS_MSG_FORMAT_E
```

● 成员

成员	描述
QL_SMS_MSG_FORMAT_GSM_7BIT	GSM 7-bit
QL_SMS_MSG_FORMAT_BINARY_DATA	二进制短信
QL_SMS_MSG_FORMAT_UCS2	UCS-2 编码短信

2.3.6. ql_sms_send_msg_async

该函数用于异步发送文本短信。

● 函数原型

```
int ql_sms_send_msg_async(ql_sms_msg_t *p_msg, int *id, ql_sms_msg_async_cb_f cb)
```

- 参数

p_msg:

[In] 文本短信；详见第2.3.5.1章。

id:

[Out] 异步发送文本短信事件 ID。用于在回调函数中标识文本短信事件。

cb:

[In] 异步发送文本短信回调函数。用于接收文本短信发送结果；详见第2.3.6.1章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败；错误码详见 *ql_type.h*。

备注

为方便用户使用，当短信息格式为 GSM 7-bit (*QL_SMS_MSG_FORMAT_GSM_7BIT*) 时，用户输入 ISO8859-1 格式的短信息即可，无需转换成 GSM 7-bit 格式，该函数将自动进行格式转换。ISO8859-1 编码向下兼容 ASCII 编码。

2.3.6.1. ql_sms_msg_async_cb_f

该回调函数接收文本短信异步发送结果。

- 函数原型

```
typedef void (*ql_sms_msg_async_cb_f)(int id, int result)
```

- 参数

id:

[In] 异步发送文本短信事件 ID。

result:

[In] 异步发送文本短信结果。

0 成功

其他值 失败；错误码详见 *ql_type.h*。

2.3.7. ql_sms_set_msg_rcv_cb

该函数用于设置文本短信接收回调函数。

- 函数原型

```
int ql_sms_set_msg_rcv_cb(ql_sms_msg_rcv_cb_f cb)
```

- 参数

cb:

[In] 文本短信接收回调函数；详见第 2.3.7.1 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败；错误码详见 *ql_type.h*。

2.3.7.1. ql_sms_msg_rcv_cb_f

该回调函数接收文本短信。

- 函数原型

```
typedef void (*ql_sms_msg_rcv_cb_f)(ql_sms_msg_t *p_msg, ql_sms_timestamp_t *timestamp, ql_sms_user_data_head_t *head)
```

- 参数

p_msg:

[In] 文本短信；详见第 2.3.5.1 章。

timestamp:

[In] 文本短信时间戳；详见第 2.3.7.2 章。

head:

[In] 长短信头部信息。用于标识长短信中的每个子短信；详见第 2.3.7.3 章。

备注

为方便用户使用，当短信息格式为 GSM 7-bit (*QL_SMS_MSG_FORMAT_GSM_7BIT*) 时，SDK 会自动将短信息转换成 ISO8859-1 格式，用户无需自己转换。ISO8859-1 编码向下兼容 ASCII 编码。

2.3.7.2. ql_sms_timestamp_t

文本短信时间戳结构体定义如下：

```
typedef struct
{
    uint8_t year;
    uint8_t month;
    uint8_t day;
    uint8_t hours;
    uint8_t minutes;
    uint8_t seconds;
    uint8_t timezone;
} ql_sms_timestamp_t
```

● 参数

类型	参数	描述
uint8_t	<i>year</i>	年。自 2000 年起计算。例如，19 表示 2019 年。
uint8_t	<i>month</i>	月
uint8_t	<i>day</i>	日
uint8_t	<i>hours</i>	小时。24 小时制。范围：0~23。
uint8_t	<i>minutes</i>	分钟
uint8_t	<i>seconds</i>	秒
uint8_t	<i>timezone</i>	时区。单位：1/4 小时。例如，中国位于+8 区，其 <i>timezone</i> 为+32。

2.3.7.3. ql_sms_user_data_head_t

长短信头部信息结构体定义如下：

```
typedef struct
{
    uint8_t valid;
    uint8_t total_seg;
    uint8_t cur_seg_num;
    uint8_t ref_num;
} ql_sms_user_data_head_t
```

● 参数

类型	参数	描述
uint8_t	<i>valid</i>	本结构体是否有效。 1 有效 0 无效，即本条短信不是长短信。
uint8_t	<i>total_seg</i>	长短信的总段数
uint8_t	<i>cur_seg_num</i>	本条短信的段数编号
uint8_t	<i>ref_num</i>	本条短信的接收编号，为本条短信的唯一标识号。

2.3.8. ql_sms_send_pdu

该函数用于发送 PDU 短信。

● 函数原型

```
int ql_sms_send_pdu(ql_sms_pdu_t *p_pdu)
```

● 参数

p_pdu:

[In] PDU 短信；详见第 2.3.8.1 章。

● 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败；错误码详见 *ql_type.h*。

2.3.8.1. ql_sms_pdu_t

PDU 短信结构体定义如下：

```
typedef struct
{
    QL_SMS_PDU_FORMAT_E format;
    int content_size;
    char content[QL_SMS_MAX_SEND_PDU_LENGTH];
} ql_sms_pdu_t
```


- 参数

类型	参数	描述
<code>QL_SMS_PDU_FORMAT_E</code>	<code>format</code>	PDU 短信格式；详见第 2.3.8.2 章。
<code>int</code>	<code>content_size</code>	PDU 短信长度。范围：0~255；单位：字节。
<code>char</code>	<code>content</code>	PDU 短信内容

2.3.8.2. QL_SMS_PDU_FORMAT_E

PDU 短信格式枚举定义如下：

```
typedef enum
{
    QL_SMS_PDU_FORMAT_CDMA = 0,
    QL_SMS_PDU_FORMAT_GW_PP = 6,
} QL_SMS_PDU_FORMAT_E
```

- 成员

成员	描述
<code>QL_SMS_PDU_FORMAT_CDMA</code>	CDMA 制式下的 PDU 短信格式（不支持）
<code>QL_SMS_PDU_FORMAT_GW_PP</code>	GSM 和 WCDMA 制式下的点对点短信消息协议的 PDU 短信格式

备注

编码 PDU 短信时，无需添加短信中心号码。

2.3.9. ql_sms_send_pdu_async

该函数用于异步发送 PDU 短信。

- 函数原型

```
int ql_sms_send_pdu_async(ql_sms_pdu_t *p_pdu, int *id, ql_sms_pdu_async_cb_f cb)
```

- 参数

p_pdu:

[In] PDU 短信；详见第 2.3.8.1 章。

id:

[Out] 异步发送 PDU 短信事件 ID。用于在回调函数中标识 PDU 短信事件。

cb:

[In] 异步发送 PDU 短信回调函数。用于接收 PDU 短信发送结果；详见第 2.3.9.1 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败；错误码详见 *ql_type.h*。

2.3.9.1. ql_sms_pdu_async_cb_f

该回调函数接收 PDU 短信异步发送结果。

- 函数原型

```
typedef void (*ql_sms_pdu_async_cb_f)(int id, int result)
```

- 参数

id:

[In] 异步发送 PDU 短信事件 ID。

result:

[In] 异步发送 PDU 短信结果。

0 成功

其他值 失败；错误码详见 *ql_type.h*。

2.3.10. ql_sms_set_pdu_rcv_cb

该函数用于设置 PDU 短信接收回调函数。

- 函数原型

```
int ql_sms_set_pdu_rcv_cb(ql_sms_pdu_rcv_cb_f cb)
```

- 参数

cb:

[In] PDU 短信接收回调函数；详见第 2.3.10.1 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败；错误码详见 *ql_type.h*。

2.3.10.1. ql_sms_pdu_rcv_cb_f

该回调函数接收 PDU 短信。

- 函数原型

```
typedef void (*ql_sms_pdu_rcv_cb_f)(ql_sms_pdu_t *p_pdu)
```

- 参数

p_pdu:

[In] PDU 短信；详见第 2.3.8.1 章。

2.3.11. ql_sms_set_service_error_cb

该函数用于设置 SMS 服务异常回调函数。

- 函数原型

```
int ql_sms_set_service_error_cb(ql_sms_service_error_cb_f cb)
```

- 参数

cb:

[In] SMS 服务异常回调函数；详见第 2.3.11.1 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败；错误码详见 *ql_type.h*。

备注

模块当前只在 SMS 服务端异常退出时才会调用注册的 SMS 服务异常回调函数。

2.3.11.1. ql_sms_service_error_cb_f

该回调函数处理 SMS 服务异常。

● 函数原型

```
typedef void (*ql_sms_service_error_cb_f)(int error)
```

● 参数

error:
[In] 错误码。详见 *ql_type.h*。

2.4. 错误码及恢复措施

常见 SMS API 的错误码及对应的恢复措施如下表所示。

表 2：错误码及恢复措施

返回值		含义	恢复措施
值	错误码		
-1003	QL_ERR_UNKNOWN	未知错误	调用 <i>ql_sms_deinit()</i> 后再调用 <i>ql_sms_init()</i> 初始化 SMS 服务后重试。若仍报错，则需抓取调用 API 全过程的 log 并提交至移远通信技术支持。
-1006	QL_ERR_UNSUPPORTED	不支持	当前操作不支持，请检查参数等是否正确，然后重试。若仍报错，则需抓取调用 API 全过程的 log 并提交至移远通信技术支持。
-1010	QL_ERR_INVALID_ARG	API 参数无效	保证 API 参数有效性后重试
-1030	QL_ERR_NO_MEM	内存不足	检查系统内存使用率，确认占用内存异常的进程是否有内存泄漏，或主动释放一些内存后重试。

-1034	QL_ERR_NOT_INIT	未做初始化	先调用 <code>ql_sms_init()</code> 初始化 SMS 服务后重试
-1067	QL_ERR_SERVICE_NOT_READY	服务未就绪	重复调用 <code>ql_sms_init()</code> ，尝试一段时间（建议 30 秒），若仍然失败，则需抓取调用 API 全过程的 log 并提交至移远通信技术支持。
-1078	QL_ERR_SMSC_ADDR	短信服务中心地址错误	设置正确的短信服务中心地址后重试。若仍报错，则需抓取调用 API 全过程的 log 并提交至移远通信技术支持。
-1099	QL_ERR_SERVICE_ABORT	服务异常退出	调用 <code>ql_sms_deinit()</code> 后再调用 <code>ql_sms_init()</code> ，待初始化 SMS 服务成功后重试。

Quectel
Confidential

3 示例

本章所述代码示例均摘自 `sample/sms/main.c`，用户可自行查看接口函数的完整示例。

请注意以下几点：

1. 运行示例文件前必须调用 `ql_sms_init()` 初始化 SMS 服务；
2. 程序退出前或不使用 SMS 时，必须调用 `ql_sms_deinit()` 释放资源。

3.1. 获取短信中心号码

```
void item_ql_sms_get_service_center_addr(void)
{
    int ret = 0;
    char addr[QL_SMS_MAX_ADDR_LENGTH] = {0};

    printf("test ql_sms_get_service_center_addr: ");
    ret = ql_sms_get_service_center_addr(addr, sizeof(addr));
    if (ret == QL_ERR_OK)
    {
        printf("ok\nservice center addr: %s\n", addr);
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}
```

3.2. 发送文本短信

如下代码片段展示了如何发送英文、二进制和中文短信。异步发送文本短信与以下实例类似，用户可自行参考。

```
void item_ql_sms_send_msg(void)
{
    int ret = 0;
    char buf[QL_SMS_MAX_SEND_MSG_LENGTH] = {0};
    int len = 0;
    int v = 0;

    ql_sms_msg_t *p_msg = calloc(1, sizeof(*p_msg));
    if (NULL == p_msg)
    {
        printf("out of memery\n");
        return;
    }

    printf("test ql_sms_send_msg: \n");
    printf("please enter msg format(0-gsm7, 1-binary, 2-ucs2): ");
    scanf("%d", (int *)&p_msg->format);
    if (2 < p_msg->format || 0 > p_msg->format)
    {
        free(p_msg);
        printf("bad format\n");
        return;
    }
    printf("please enter phone number: ");
    scanf(" %s", p_msg->addr);
    getchar();
    printf("example binary content: 30 31 32 33 q\n");
    printf("please enter content(binary content(hex) end with `q`): ");
    switch (p_msg->format)
    {
        case QL_SMS_MSG_FORMAT_GSM_7BIT:
        case QL_SMS_MSG_FORMAT_UCS2:
            fgets(buf, QL_SMS_MAX_SEND_MSG_LENGTH, stdin);
            len = strlen(buf);
            if ('\r' == buf[len-1] || '\n' == buf[len-1])
            {
                buf[len-1] = 0;
                len--;
            }
            break;
    }
}
```

```

    }
    break;
case QL_SMS_MSG_FORMAT_BINARY_DATA:
    while (1 == scanf("%x", &v))
    {
        buff[len++] = v;
    }
    getchar(); // read `q'
    getchar(); // read `n'
    break;
default:
    free(p_msg);
    return;
}
printf("raw content size: %d\n", len);
if (QL_SMS_MSG_FORMAT_UCS2 == p_msg->format)
{
    p_msg->content_size = str_utf8_to_ucs2((uint8_t *)buff, len, (uint8_t *)p_msg->content);
}
else
{
    memcpy(p_msg->content, buff, len);
    p_msg->content_size = len;
}
printf("sending msg, %d bytes...", p_msg->content_size);
ret = ql_sms_send_msg(p_msg);
if (ret != QL_ERR_OK)
{
    printf("failed, ret = %d\n", ret);
}
else
{
    printf("ok\n");
}

free(p_msg);
}

```

3.3. 接收文本短信

文本短信接收回调函数代码片段如下：


```
static void sms_msg_rcv_cb(
    ql_sms_msg_t *p_msg,
    ql_sms_timestamp_t *timestamp,
    ql_sms_user_data_head_t *head)
{
    if (NULL == p_msg)
    {
        return;
    }
    printf("\nnew message coming...\n");
    printf("who: %s\n", p_msg->addr);
    if (NULL != timestamp)
    {
        printf("timestamp: %02hhu-%02hhu-%02hhu %02hhu:%02hhu:%02hhu+%hhhu\n",
            timestamp->year,
            timestamp->month,
            timestamp->day,
            timestamp->hours,
            timestamp->minutes,
            timestamp->seconds,
            timestamp->timezone);
    }
    printf("content size: %d\n", p_msg->content_size);
    printf("content: ");
    switch (p_msg->format)
    {
        case QL_SMS_MSG_FORMAT_GSM_7BIT:
            printf("%s\n", p_msg->content);
            break;
        case QL_SMS_MSG_FORMAT_BINARY_DATA:
        {
            int i = 0;
            for (i = 0; i < p_msg->content_size; i++)
            {
                printf("%02x ", p_msg->content[i]);
            }
            printf("\n");
        }
        break;
        case QL_SMS_MSG_FORMAT_UCS2:
        {
            char *buf = calloc(1, QL_SMS_MAX_RECV_MSG_LENGTH * 2);
            if (NULL == buf)
            {

```

```

        return;
    }
    unicode_str_to_utf8_str((unsigned short *)p_msg->content, (uint8_t *)buf,
QL_SMS_MAX_RECV_MSG_LENGTH * 2);
    printf("%s\n", buf);
    free(buf);
}
break;
}
if (NULL != head && 1 == head->valid)
{
    printf("total segments: %d\n", head->total_seg);
    printf("current segment: %d\n", head->cur_seg_num);
    printf("reference number: %d\n", head->ref_num);
}
}
}

```

文本短信接收函数代码片段如下：

```

void item_ql_sms_set_msg_rcv_cb(void)
{
    int ret = 0;

    printf("test ql_sms_set_msg_rcv_cb: ");
    ret = ql_sms_set_msg_rcv_cb(sms_msg_rcv_cb);
    if (ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}
}

```

接收文本短信结果展示如下：

```

new message coming...
who: +8613670949352
timestamp: 21-10-12 11:18:10+32
content size: 13
content: 30 31 32 33 q

```

3.4. 发送 PDU 短信

异步发送 PDU 短信与以下示例类似，用户可自行参考。

```
void item_ql_sms_send_pdu(void)
{
    int ret = 0;
    int v = 0;
    ql_sms_pdu_t pdu = {0};

    printf("test ql_sms_send_pdu: \n");
    printf("please enter pdu format(0-CDMA, 6-GW-PP): ");
    scanf("%d", (int *)&pdu.format);
    if (QL_SMS_PDU_FORMAT_CDMA != pdu.format && QL_SMS_PDU_FORMAT_GW_PP !=
pdu.format)
    {
        printf("bad format: %d\n", pdu.format);
        return;
    }

    getchar();
    printf("example GW-PP pdu content: 00 01 00 0b 81 71 26 21 53 46 f7 00 04 0A 30 31 32 33 34 35
36 37 38 39 q\n");
    printf("please enter content(hex), end with `q': ");
    while (1 == scanf("%x", &v))
    {
        pdu.content[pdu.content_size++] = v;
    }
    getchar(); // read `q'
    getchar(); // read '\n'

    printf("content size: %d\n", pdu.content_size);
    printf("sending pdu...");
    ret = ql_sms_send_pdu(&pdu);
    if (ret != QL_ERR_OK)
    {
        printf("failed, ret = %d\n", ret);
    }
    else
    {
        printf("ok\n");
    }
}
```

3.5. 接收 PDU 短信

PDU 短信接收回调函数代码片段如下：

```
static void sms_pdu_rcv_cb(ql_sms_pdu_t *p_pdu)
{
    int i = 0;

    printf("\nnew pdu coming...\n");
    if (NULL == p_pdu)
    {
        return;
    }
    printf("format: ");
    switch (p_pdu->format)
    {
        case QL_SMS_PDU_FORMAT_CDMA:
            printf("cdma\n");
            break;
        case QL_SMS_PDU_FORMAT_GW_PP:
            printf("gw-pp\n");
            break;
        default:
            printf("unknown\n");
            break;
    }
    printf("content size: %d\n", p_pdu->content_size);
    printf("content(hex): ");
    for (i = 0; i < p_pdu->content_size; i++)
    {
        printf("%02x ", p_pdu->content[i]);
    }
    printf("\n");
}
```

PDU 短信接收函数代码片段如下：

```
void item_ql_sms_set_pdu_rcv_cb(void)
{
    int ret = 0;

    printf("test ql_sms_set_pdu_rcv_cb: ");
    ret = ql_sms_set_pdu_rcv_cb(sms_pdu_rcv_cb);
}
```

```

if (ret == QL_ERR_OK)
{
    printf("ok\n");
}
else
{
    printf("failed, ret = %d\n", ret);
}
}

```

接收 PDU 短信结果展示如下：

```

Format: gw-pp
content size: 37
content(hex): 08 91 68 31 08 70 75 05 f0 04 0d 91 68 31 76 90 94 53 f2 00 08 12 01 41 51 54 95 23 08 00 71 00 71 00 71 00 71
new message coming from sim_id:0...
who: +8613670949352
timestamp: 21-10-14 15:45:59+32
content size: 8
content: qqqq

```

4 附录 参考文档及术语缩写

表 3: 参考文档

文档名称
[1] Quectel_AG35-CET&AG35-EUT_QuecOpen(SDK)_快速开发指导

表 4: 术语缩写

缩写	英文描述	中文描述
API	Application Programming Interface	应用程序编程接口
ASCII	American Standard Code for Information Interchange	美国信息交换标准代码
CDMA	Code-Division Multiple Access	码分多址
GSM	Global System for Mobile Communications	全球移动通信系统
IoV	Internet of Vehicles	车联网
ISO	International Standard Organization	国际标准化组织
PDU	Protocol Data Unit	协议数据单元
SDK	Software Development Kit	软件开发工具包
SMS	Short Message Service	短信息服务
UCS	Universal Character Set	通用字符集
WCDMA	Wideband Code Division Multiple Access	宽带码分多址